



BUILDING A FLASHCARD API FROM ZERO — OUTLINE STUDY PODCAST

FastAPI, Supabase & Multi-User Architecture — Companion Audio Guide

Welcome. You should have the printed outline in front of you — we'll be working through it together. Not reading it aloud — actually working through it. When there's a table or a diagram worth looking at, we'll tell you. When we're just talking through ideas, you can sit back and listen. Let's begin at the beginning: Section I.

SECTION I — WHY FASTAPI AND WHAT PROBLEM DOES IT SOLVE

❑ So we're building a flashcard API. Before we touch any code — why do we even need an API? What problem are we actually solving?

Here's the situation. You're a Python developer. You've written a script that reads flashcard data from a JSON file, processes it beautifully, prints it to the terminal. It works. It's done. You're happy. Now someone says: "I want to use your flashcards on my phone." And you hit a wall. Because your script runs, does its thing, and exits. There is no door. There is no way for an outside device — a phone, a browser, another machine — to say "hey, give me those flashcards." The script is a closed room.

❑ Okay, but couldn't we just put the JSON file on a web server? Then anyone could download it.

That's the obvious next thought, and it's worth understanding why it fails. A static file on a web server solves exactly one problem — retrieval. Anyone can grab the file. Great. But now try to create a new flashcard. Try to edit one. Try to delete one. You can't. The file is inert. It doesn't know what kind of request you're sending. It doesn't care. It just hands you the same content every time. If you look at the outline, subsection A lays this out with three points — the limitation of scripts, the insufficiency of static files, and then the solution. That solution is what we need to talk about.

❑ Which is an API.

Right. An API — Application Programming Interface — is a program that stays alive. It sits on a server, permanently listening. When a request comes in, it figures out what the request wants — are you reading data? Creating something? Updating? Deleting? — and it does the work. The outline has a great analogy here: think of a receptionist at a front desk. The filing cabinets are your database. Without the receptionist, the cabinets exist but nobody from outside can interact with them. The API is that receptionist — always present, always ready.

❑ Alright, so an API listens for requests. But how does it know what I'm asking for? If I say "flashcards" — do I want all of them? Just one? Do I want to create a new one?

Two things work together here. First, the endpoint — that's a specific URL, like localhost colon 8000 slash cardfolders. Different URLs point to different resources. Second, the HTTP method — that's the verb attached to your request. And this is where it gets elegant. Flip to subsection B in your outline — there's a table mapping HTTP methods to CRUD operations.

❑ I see it. GET, POST, PUT, DELETE.

Exactly. GET means read — just retrieve data, don't touch anything. POST means create — I'm sending you new data, store it. PUT means update — here's a modified version of something that already exists. DELETE means remove it. Four verbs, four operations. And those four operations — Create, Read, Update, Delete, the acronym CRUD — are the foundation of virtually every data-driven application you'll ever build. Every app you use daily — your email, your social media, your banking — is fundamentally CRUD operations on a database, exposed through an API.

❑ That's a clean mapping. So now the question is — what tool do we use to build this API? Python has options, right?

It does, and this is worth thinking through. If you look at subsection C in the outline, it lists three major Python frameworks. Let me explain why two of them lose.

Flask is the oldest. It's a micro-framework — gives you the absolute bare minimum, and you build everything else yourself. Need data validation? Write it by hand. Need documentation? Install a plugin and configure it. It's like getting an empty room and being told to build your own furniture.

Django REST Framework is the opposite extreme. It's built on top of Django, which is a full-featured web framework. It's powerful, but it comes with an enormous amount of machinery — an ORM, an admin panel, a template system. For a focused API project, it's like renting a warehouse when you need a workshop.

❑ So FastAPI sits in between?

Not exactly between — it takes a fundamentally different approach. FastAPI was designed in 2018, decades after Flask and Django, and it was built to exploit modern Python features that didn't exist when those older frameworks were designed. Here's what that means in practice.

First benefit — automatic data validation. FastAPI uses a library called Pydantic — the name's a play on "pedantic" — that lets you define the exact shape of data you expect. You say: this field must be a string, this one must be an integer, this one is optional. When a request arrives that doesn't match? Pydantic rejects it instantly with a detailed error message — a 422 Unprocessable Entity. You write zero validation code. In Flask, you'd write all of that by hand — check every field, check its type, check if it's present. That's tedious, error-prone work that FastAPI eliminates completely.

❑ So the framework guards your door for you.

Exactly. Bad data never touches your database. It's caught at the boundary.

Second benefit — automatic interactive documentation. The moment your FastAPI server starts, it generates a fully interactive documentation page at slash docs. You can see every endpoint, what data each one expects, and — this is the key — you can test them right in your browser. Click "Try it out," fill in some data, hit execute. You don't need Postman, you don't need curl, you don't need any external tool during development. It builds itself from your code.

❑ Wait — it generates docs from your code? How?

That's the third benefit, and it's the cleverest part. FastAPI uses Python's native type hint system — introduced in Python 3.5 — for a dual purpose. When you write something like `front underscore text underscore 1 colon str`, that single annotation does two things simultaneously: it tells Pydantic to validate that the value is a string, AND it tells the documentation generator to show it as a string field. One line of code, two jobs done. In Flask, validation and documentation are separate systems that you maintain independently.

There's also async support — FastAPI can handle many requests concurrently without blocking — but we're using synchronous code in this tutorial for simplicity. The capability's there when you need it.

❑ Okay. So we have our framework. Now we need a database. The outline mentions Supabase — why not just use a local database like SQLite?

SQLite would work — for exactly one person, on exactly one machine. It stores everything in a single file on your local disk. That's fine for a personal script, but remember our original problem: external clients need to access the data. A phone in another city can't read a file on your laptop.

Supabase solves this by giving you a hosted PostgreSQL database — that's one of the most powerful relational databases in the world, used by Instagram, Spotify, Netflix. But Supabase wraps it in a friendly interface. You get a visual web dashboard where you can create tables, inspect data, run queries — all through your browser. And the Python client library is clean — you write things like `supabase dot table cards dot select star dot execute`. Very readable, very consistent.

The real kicker? When you eventually deploy your API to a production server, the Supabase connection works identically. No migration, no reconfiguration. It's the same code.

□ That's three advantages if you're looking at the outline — remote accessibility, visual dashboard, clean Python client. Now, subsection E is big. It talks about four features that transform what we're building from a toy into a platform. What are these four features?

This is where the project gets interesting, so let me build the picture. Imagine we've built a simple flashcard API — one user, no security, no access control. It works. Now imagine two people use it. Immediately, we have problems.

Problem one: if User A creates a card set called "Spanish Verbs," can User B delete it? Of course not — but nothing in a naive system prevents it. We need ownership. Every folder, every card set must know who created it, and every write operation — create, update, delete — must check: are you the owner? If not, you're blocked.

□ So that's feature one — ownership.

Right. Feature two is visibility. Say you've built a great card set and you want to share it publicly so anyone can browse it. But you also have private card sets — personal study notes you don't want others to see. Each card set needs a flag — is underscore public — that defaults to false. Private unless you explicitly say otherwise.

□ Makes sense. What about tracking changes?

That's feature three — audit trails. If someone edits a flashcard and the text gets corrupted, you want to know when it happened and who did it. Every card modification gets stamped with a timestamp — updated underscore at — and the identity of the person who made the change — updated underscore by. A permanent record.

❑ And feature four?

Feature four is copying — and this is the one that really makes the platform useful. Imagine a Spanish teacher builds a beautiful 200-card set and shares it publicly. A student finds it, loves it, but wants to personalise it — add notes, remove cards they already know, tweak translations. The student can't edit the original — they're not the owner, and even if they could, their changes would affect every other student. So the API supports a deep copy — a complete, independent clone into the student's own account. Their copy, their rules. The original untouched.

❑ That's a nice set of features. Let me look at the section summary in the outline... it ties it together: static files can't handle CRUD, FastAPI was chosen for validation, documentation, and type hints, Supabase for remote hosting and the dashboard, and these four multi-user features — ownership, visibility, audit trails, copying. And eight sections ahead to build it all. Let's move on.

SECTION II — ENVIRONMENT SETUP ON macOS

□ Section II — setting up the environment. Before we build anything, we need the workshop ready. The outline says there are four steps, and they must happen in order. Why does order matter?

Because each step depends on the previous one. Think of it as: first build the walls — that's your virtual environment, the isolation. Then bring in the tools — that's installing the packages. Then lock up the valuables — that's your credentials file and the gitignore. Then test the lights — that's your verification run. If you try to install packages before the environment exists, they go into the wrong place. If you skip the gitignore and push to GitHub, your credentials are exposed. The sequence is deliberate.

□ Step one — the project folder and virtual environment. What do we actually type?

Three commands. `mkdir flashcard-api` — creates the folder. `cd flashcard-api` — go inside. `workon vintel` — activate the virtual environment. After that last one, your terminal prompt will change — you'll see `vintel` in parentheses at the beginning. That's your confirmation.

□ Now, I know what a virtual environment is in general terms, but why does it matter here specifically?

Think about what happens without one. You install FastAPI for this project. Next month, you start a different project that needs an older version of one of FastAPI's dependencies. You install it, and it overwrites what the flashcard project needs. Now your flashcard project is broken, and you didn't touch it. Dependency hell.

A virtual environment is a sealed room. We're using `virtualenvwrapper`, so the `workon` command activates a named environment stored centrally — typically in the `.virtualenvs` folder in your home directory. When you install libraries while inside, they go into that environment only. Not your system Python, not any other project. Complete isolation. Your project folder stays clean.

❑ One thing the outline emphasises — and this catches people — is that the activation doesn't persist.

Right. Every time you open a new terminal window, you're starting fresh. No virtual environment active. You have to run `workon vintel` again. Every single time. Forget this and you'll be installing packages into your system Python wondering why nothing works.

❑ Step two — installing packages. One command?

One command: `pip install fastapi uvicorn supabase python-dotenv`. Four packages. But let me explain what each one actually does, because they're not interchangeable — they do fundamentally different jobs.

❑ Let's start with fastapi itself.

FastAPI is the framework — the rule book. It gives you decorators like `@app.get` and `@app.post` that turn ordinary Python functions into API endpoints. It handles routing — figuring out which function to call based on the URL. It handles parsing — extracting data from the request. But here's what trips people up: FastAPI cannot listen for HTTP requests on its own. It's not a server. It's a set of rules and decorators. It needs an engine.

❑ And that engine is uvicorn?

Exactly. Uvicorn is an ASGI server — Asynchronous Server Gateway Interface. Think of ASGI as a contract. FastAPI speaks ASGI, Uvicorn speaks ASGI, so they can work together. Uvicorn is the piece that actually listens on a network port — port 8000 by default — receives incoming HTTP requests from the outside world, hands them to FastAPI for processing, and sends the responses back. FastAPI defines what to do. Uvicorn is how requests arrive and depart. Two separate jobs, two separate packages.

❑ What about the supabase package?

That's the official Python client for talking to your hosted database. It gives you that clean chained syntax — `supabase.table('cards').select('star').execute()`. Nothing fancy, just the bridge between your Python code and the PostgreSQL database sitting in the cloud.

❑ And python-dotenv — what's that for?

This is about keeping secrets secret. Your Supabase URL and API key are credentials. If you hardcode them in your Python files and push to GitHub, anyone can see them and access your database. `python-dotenv` reads a separate dot env file — a simple text file with key-value pairs — and loads them as environment variables. Your code reads os dot environ bracket SUPABASE underscore URL bracket. The credentials never appear in source code.

❑ Which brings us to step three — setting up that dot env file. What goes in it?

Two lines. SUPABASE underscore URL equals your project URL. SUPABASE underscore KEY equals your anon public key. You find both by going to your Supabase dashboard — supabase dot com, open your project, Project Settings in the left sidebar, then click API. Both values are right there. Copy them in.

❑ And the gitignore — the outline says we need to protect certain files from version control.

The dot gitignore file tells Git: never track these, never upload these to GitHub, even by accident. We exclude the dot env file — your credentials. We exclude double underscore pycache double underscore — that's Python's auto-generated bytecode cache, not source code. And we exclude venv slash — in case a local virtual environment folder exists. You don't want any of these in your repository.

❑ Step four — verification. How do we know everything works?

Create a tiny test file — test underscore setup dot py. Import FastAPI, create an app instance, add one GET endpoint at slash that returns a JSON message. Then run `uvicorn test underscore setup colon app dash dash reload`.

❑ What does that command format mean? test underscore setup colon app?

It's filename without extension, colon, variable name. So Uvicorn looks inside test underscore setup dot py for an object called app. The dash dash reload flag is a development convenience — Uvicorn watches your files and automatically restarts the server whenever you save a change. You never have to manually stop and restart during development. Without it, you'd be hitting Control C and re-running the command every time you change a line.

Open your browser to localhost colon 8000 — you should see the JSON message. Then go to localhost colon 8000 slash docs — there's the Swagger documentation with one endpoint listed. Control C to stop the server, delete the test file. It served its purpose.

☐ Let me check the section summary in the outline... project folder, four packages installed, dot env and dot gitignore created, verification done. Clean starting point. Shall we build our database?

SECTION II — COMPLETE CODE

Terminal commands — project setup:

```
mkdir flashcard-api
cd flashcard-api
workon vintel
```

Terminal commands — install packages:

```
pip install fastapi uvicorn supabase python-dotenv
```

File: .env

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your-anon-key-here
```

File: .gitignore

```
.env
__pycache__/
```

File: test_setup.py (verification — delete after testing)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Flashcard API is alive"}
```

Terminal commands — run verification:

```
uvicorn test_setup:app --reload
```

SECTION III — DATABASE DESIGN IN SUPABASE

❑ Section III — database design. This is where we decide the shape of our data. Before we look at specific tables, let me ask a foundational question: why not just one big table? Why can't I put everything — users, folders, card sets, cards — into a single table?

That's the right question to start with, and I want you to really feel the pain before I give you the solution. Imagine one giant table. Every row is a card. And each row contains the card's content, plus the name of the card set it belongs to, plus the name of the folder, plus the user's name and email. All in one row.

Now — the folder "Spanish Vocabulary" has 500 cards across 10 sets. The string "Spanish Vocabulary" is stored 500 times. Five hundred identical copies. Want to rename the folder? Update 500 rows. Miss one? Now you have 499 rows saying "Spanish Language" and one still saying "Spanish Vocabulary." Which one is correct? There's no authoritative source — each row is a separate copy that can drift.

❑ That's the inconsistency problem.

And it gets worse. Say you delete all the cards in a particular set. The set itself vanishes — because it only existed as data repeated in card rows. There's no independent record of it. The user might have wanted to keep the empty set as a placeholder, but it's gone. That's a deletion anomaly.

And then there's user data. Without a separate users table, every card row repeats the user's name and email. User changes their email? Every card they ever touched needs updating.

❑ Okay, I feel the pain. What's the proper solution?

Normalisation. The idea — formalised by Edgar F. Codd in 1970 — is beautifully simple: store each fact exactly once. Instead of writing "Spanish Vocabulary" 500 times, store it once in a cardfolders table and give it a unique ID. Every card stores just that ID — a lightweight pointer. Need to rename? Change one row. Every reference to it automatically reflects the new name.

❑ So we split into separate tables. How many do we need?

Four. And if you look at the table in subsection A of your outline, you'll see them laid out with their dependencies. Users — that's identity, who uses the system. Depends on nothing, created first. Cardfolders — containers for card sets. Depends on users. Cardsets — the actual collections, with ownership and visibility. Depends on both users and cardfolders. Cards — the individual flashcards with audit trails. Depends on cardsets and users for audit.

❑ Let me trace the hierarchy. Users own folders. Folders contain card sets. Card sets contain cards. It's a strict chain.

Exactly. And that chain matters for creation order in the database — you can't reference a table that doesn't exist yet. Users first, then folders, then card sets, then cards.

❑ Now, these tables need to be connected somehow. The outline talks about foreign keys — what exactly is a foreign key?

Let's build from first principles. Every table has a primary key — a column that uniquely identifies each row. In our design, it's always the id column, a UUID. No two rows can have the same one.

A foreign key is a column in a different table that points to a primary key. The cardfolders table has a user underscore id column. Each value in that column is a UUID that matches an id in the users table. That's the link — "this folder belongs to this user." And here's the crucial part: the database enforces this. You literally cannot insert a value in user underscore id that doesn't correspond to an existing user. The database rejects it.

❑ So the relationships are enforced, not just suggested. Now — what happens when you delete a parent? If I delete a user, what happens to their folders?

This is one of the most important design decisions in the whole project, and the outline lays out two options at subsection C point two. Let me give you both, and you'll immediately see why each is used where it is.

Option one: CASCADE. When you delete the parent, all children are automatically deleted. The logic is: if the child cannot meaningfully exist without the parent, destroy them together. A folder without an owner makes no sense. A card set without a folder makes no sense. A card without a card set makes no sense. So: delete a user, their folders cascade away. Delete a folder, its card sets cascade. Delete a card set, its cards cascade.

□ That's clean. What's option two?

Option two: SET NULL. When you delete the referenced entity, the foreign key column is set to null — but the child survives. The logic is: the reference is informational, not existential. Think about the audit trail. A card has an updated underscore by field pointing to the user who last edited it. If that user's account is deleted, should the card be destroyed? Absolutely not — it belongs to someone else. It's perfectly valid. We just lose the information about who made that particular edit. The field becomes null: "the person who last edited this card no longer exists."

□ Oh — so CASCADE is for existential relationships and SET NULL is for informational references. That's a clean distinction.

And it's consistent throughout the whole design. If you look at the table in your outline — four rows. User deleted, folders cascade. Folder deleted, card sets cascade. Card set deleted, cards cascade. But user deleted and the updated underscore by field on cards they edited — SET NULL. The card lives on. Only the editor reference clears.

□ Let me ask about the multi-user columns specifically. The outline has three categories in subsection D.

Right. Category one — ownership. Both cardfolders and cardsets carry a user underscore id column pointing to the users table. Every write operation checks this column: does the requester match the owner? If not — 403 Forbidden.

Now here's a subtle design choice. Cards don't have their own user underscore id. Card ownership is inherited from the parent card set. If you own the set, you own the cards inside it. This avoids duplicating ownership data at the card level.

❑ Category two — visibility?

The cardsets table has an `is_public` boolean, defaulting to false. Private unless you explicitly flip it. And notice — this lives at the card set level, not the folder level.

❑ Why not the folder level? That seems like it would be simpler.

Think about what happens. You have a folder called "Languages" with three card sets inside — "Spanish Basics," "French Verbs," and your private "Embarrassing Mistakes." If visibility is on the folder, it's all-or-nothing. You either share everything in the folder or nothing. But if visibility is on the card set, you can make "Spanish Basics" public, keep "Embarrassing Mistakes" private, and there's no contradiction. Each set is independently controlled.

Even worse — if you had a public card set inside a private folder, what should happen? Is it visible or not? The rules conflict. Keeping visibility on card sets avoids all that ambiguity.

❑ Category three — the audit columns.

On the cards table only. `updated_at` — a `timestampz`, starts as null. `updated_by` — a `uuid` pointing to users, also starts null, with `SET NULL` on delete. When a card is modified, both get stamped. A null `updated_at` clearly communicates: this card has never been edited since creation. It's not "we don't know when it was edited." It's "it hasn't been."

❑ Now let's actually create these tables. The outline has a step-by-step in subsection E. First — how do we navigate the Supabase dashboard?

When you log into `supabase dot com` and open your project, the left sidebar has a bunch of options — Authentication, Database, Storage, and more. The one you want is Table Editor. It gives you a graphical interface for creating tables without writing a line of SQL. Click New Table. And one important note right away: uncheck "Enable Row Level Security." We're handling security at the API level in this tutorial, not at the database level.

❑ Let's go table by table. The outline has detailed column tables for each one — I'm looking at them now. Table one: users.

Four columns. The id — type uuid, primary key, not null, with a default of gen underscore random underscore uuid. That function generates a UUID automatically for every new row. UUID stands for Universally Unique Identifier — a 128-bit number, 36-character string. The odds of two colliding are astronomically small.

Then username — text, not null, and crucially, unique. Email — same thing, text, not null, unique. The UNIQUE constraint means the database itself rejects duplicate usernames or emails. To set it in Supabase, click the gear icon on the column and check "Is Unique."

Finally, created underscore at — type timestamptz, with a default of now. Timestamptz is PostgreSQL's timestamp type that includes timezone information, so timestamps are consistent regardless of where the server is.

❑ Table two — cardfolders. I can see it depends on users.

Five columns. The id — same UUID pattern. User underscore id — uuid, not null, with a foreign key pointing to users dot id, ON DELETE CASCADE. This is the ownership link. Name — text, not null. Description — text, nullable, because not every folder needs a description. Created underscore at — timestamptz with now default.

To create that foreign key in Supabase: click the link icon next to user underscore id, select the users table and id column, and set ON DELETE to CASCADE.

❑ Table three — cardsets. Two foreign keys this time.

Seven columns. Id — UUID primary key. Cardfolder underscore id — points to cardfolders dot id with CASCADE. User underscore id — points to users dot id with CASCADE. Is underscore public — boolean, not null, default false — remember, private by default. Name — text, not null. Description — nullable text. Created underscore at — timestamptz.

❑ And table four — cards. This is the big one. If you're following along in the outline, there's a twelve-row column table.

Let me break this into groups. First, the structural columns. Id — UUID primary key. Cardset underscore id — uuid, not null, foreign key to cardsets dot id with CASCADE.

Second, the content columns — and there are eight of them, all text, all nullable. Why nullable? Because not every card uses every field. Some might only have text, no image. Some might have audio but no image. So: front underscore text underscore 1, front underscore audio underscore text underscore 1, front underscore audio underscore 1, front underscore image. And the same four for the back of the card.

❑ That's a lot of fields. Why separate audio text from display text?

Great question — we'll dig deep into that in Section VII. Short version: what looks good on screen sounds terrible when read aloud by a text-to-speech engine. The display text might say "El gato (the cat) [m.]" — perfectly clear visually. But a TTS engine would read the brackets and parentheses aloud. Garbled nonsense. The audio text field holds a cleaned version: just "El gato."

❑ Makes sense. What about the audit columns?

Third group — created underscore at with a now default. Updated underscore at — timestamptz, nullable, and critically, no default. It starts as null, meaning "never edited." Not "unknown" — actively communicating "no edits have occurred." And updated underscore by — uuid, nullable, foreign key to users dot id with ON DELETE SET NULL. Not CASCADE — because if the editor's account is deleted, the card itself should survive. Only the reference clears.

❑ The outline mentions a complete cascade chain. Walk me through what happens when you delete a user.

It's like dominoes. Delete the user — CASCADE to their folders — CASCADE to the card sets inside those folders — CASCADE to the cards inside those sets. Everything that user owned is gone. But — and this is the critical nuance — cards that the user merely edited, belonging to other users' sets, those survive. Only the updated underscore by field on those cards is set to null. The card stays. The edit record becomes "an unknown user made this change."

❑ Let me check the section summary... four tables in a strict hierarchy, normalisation eliminates duplication, foreign keys with CASCADE enforce integrity, is underscore public lives on cardsets not folders, audit trail on cards, SET NULL preserves cards when editors leave. Solid foundation. What's next — how do we organise the code?

SECTION IV — PROJECT ARCHITECTURE

❑ Section IV — project architecture. We've got four database tables. We've got a framework. Now we need to organise the actual code. Could we just write everything in one Python file?

You could — and for a tiny project with two endpoints, it would be fine. But we're building 22 endpoints. Plus ownership checks. Plus visibility filters. Plus utility functions. In one file, you'd be scrolling through hundreds of lines to find anything. Change something for card sets and you might accidentally break something for users. Multiple developers can't easily work on different parts. One file does not scale.

❑ So what's the alternative?

FastAPI has a built-in solution called `APIRouter`. A router is essentially a mini-application that handles a subset of endpoints. You create one router for user operations, one for folders, one for card sets, one for cards. Each lives in its own file. Then your main application file plugs them all in with `app.include_router`.

Think of it like a book. Each chapter covers one topic — that's a router file. The table of contents — that's `main.py`. When you need to work on card set endpoints, you open `cardset_router.py`. Nothing else is in your way.

❑ The outline has a directory tree diagram in subsection B. I'm looking at it now. Walk me through the structure.

At the top level, your `flashcard-api` folder. Inside it, four things directly: `.env` for credentials, `.gitignore`, `main.py` as the entry point, and `database.py` for the Supabase connection.

Then three subfolders. The `models` folder holds Pydantic models — the data shapes for each entity. Four files: `user_models.py`, `cardfolder_models.py`, `cardset_models.py`, `card_models.py`.

The routers folder holds the endpoint logic — same four entities, four files: user underscore router, cardfolder underscore router, cardset underscore router, card underscore router.

And a utils folder with one file: audio underscore text dot py — the regex-based audio text generator.

❑ I notice each folder has an init file — double underscore init double underscore dot py. What are those?

This is a Python convention that trips people up because it seems like magic. By default, a directory in Python is just a directory — it has no special meaning. But the moment you place an empty file called double underscore init double underscore dot py inside it, Python treats that directory as a package. That means you can import from it using dot notation — from routers import cardfolder underscore router. Without that empty file, the import fails. The file itself can be completely empty — just its presence changes the behaviour. Strange but essential.

❑ The outline mentions two foundational files in subsection C — database dot py and main dot py. What do they do?

Database dot py is the single connection point to Supabase. Four steps: import os, import load underscore dotenv, import create underscore client from supabase. Call load underscore dotenv to read the dot env file. Pull the URL and key from environment variables. Create the client with supabase equals create underscore client of url and key. That supabase object gets imported by every router — one shared connection point for the entire application.

Main dot py is the orchestrator. It creates the FastAPI instance — app equals FastAPI with a title, description, and version. Then four calls to app dot include underscore router, one per router. And one root endpoint at slash that returns a simple "Flashcard API is running" — a health check to confirm the server is alive. You start it with uvicorn main colon app dash dash reload.

❑ The section summary notes something interesting — the user underscore router and user underscore models files are entirely new compared to a single-user version of this system. Everything else existed before; the multi-user dimension adds those two files. Let's move to the models.

SECTION IV — COMPLETE CODE

Project directory structure:

```
flashcard-api/
├── .env
├── .gitignore
├── main.py
├── database.py
├── models/
│   ├── __init__.py
│   ├── user_models.py
│   ├── cardfolder_models.py
│   ├── cardset_models.py
│   └── card_models.py
├── routers/
│   ├── __init__.py
│   ├── user_router.py
│   ├── cardfolder_router.py
│   ├── cardset_router.py
│   └── card_router.py
├── utils/
│   ├── __init__.py
│   └── audio_text.py
```

Terminal commands to create the structure:

```
mkdir models routers utils
touch main.py database.py
touch models/__init__.py models/user_models.py models/cardfolder_models.py
models/cardset_models.py models/card_models.py
touch routers/__init__.py routers/user_router.py routers/
cardfolder_router.py routers/cardset_router.py routers/card_router.py
touch utils/__init__.py utils/audio_text.py
```

File: database.py

```

import os
from dotenv import load_dotenv
from supabase import create_client, Client

load_dotenv()

url: str = os.environ["SUPABASE_URL"]
key: str = os.environ["SUPABASE_KEY"]

supabase: Client = create_client(url, key)

```

File: `main.py`

```

from fastapi import FastAPI
from routers import user_router, cardfolder_router, cardset_router,
card_router

app = FastAPI(
    title="Flashcard API",
    description="Multi-user API for managing flashcard folders, sets, and
cards with ownership, public sharing, and copy features",
    version="2.0.0"
)

app.include_router(user_router.router)
app.include_router(cardfolder_router.router)
app.include_router(cardset_router.router)
app.include_router(card_router.router)

@app.get("/")
def root():
    return {"message": "Flashcard API is running"}

```

SECTION V — PYDANTIC MODELS

❑ Section V — Pydantic models. We mentioned Pydantic earlier as a validation library. But before we look at the specific models, I want to understand the problem they solve. What happens without them?

Picture this. A client sends a POST request to create a flashcard. The data arrives as JSON in the request body. JSON is just text — it could contain anything. A number where a string should be. A missing required field. Extra fields that shouldn't be there. Without validation, your server blindly tries to insert whatever arrives into the database. Best case, the database throws an error. Worst case, the bad data goes in silently and you have corruption you don't know about.

❑ So Pydantic is the bouncer at the door.

Exactly. You define Python classes that describe the exact shape of acceptable data. FastAPI uses them to validate every incoming request before your endpoint code even runs. If the data is bad — automatic 422 response with a detailed error message listing exactly which fields failed and why. If the data is good — your code gets a clean, type-safe Python object. No guessing, no checking.

❑ The outline describes three categories of models in subsection B. Create, Update, and Response. Why three? Why not one model per entity?

Because different operations need different fields, and getting this wrong creates security holes. Let me explain each category and you'll see why they must be separate.

Create models — what the client sends when making something new. These exclude auto-generated fields like id and created underscore at. The database handles those — the client has no business providing them. And crucially, they exclude user underscore id. Ownership comes from the request header, not the body. If user underscore id were in the create model, a malicious client could say "create this folder on behalf of User B." The server-side header assignment prevents that.

❑ So the Create model is deliberately missing fields. What about Update?

Update models define only what can be changed after creation. And every field is optional — the client might only want to change the name, not the description. Some fields are excluded entirely. You can't change a card set's owner — that would be transferring ownership. You can't move a card set to a different folder via update. Those exclusions are deliberate constraints.

❑ And Response models?

Response models are the complete picture — everything the client should see. They include the auto-generated id, the created underscore at timestamp, and for cards, the audit fields — updated underscore at and updated underscore by. These are the models used for what the API sends back.

❑ Let me look at the specific models in subsection C. The outline has tables for each entity. Starting with users — seems simple?

The simplest. UserCreate has just username, a required string, and email, a required string. UserResponse adds id and created underscore at. UserUpdate is not implemented — a tutorial simplification, we don't support changing usernames or emails.

❑ Card folder models?

CardfolderCreate has name, required, and description, optional, defaulting to None. No user underscore id — set from the header. CardfolderResponse adds id, user underscore id, name, description, created underscore at. Now the client can see who owns the folder.

❑ Card set models — this is where it gets interesting.

Three models. CardsetCreate has cardfolder underscore id — which folder does this set go in — name, description, and is underscore public defaulting to false. Private by default. Still no user underscore id — from the header.

CardsetUpdate has name, description, and is underscore public — all optional. Look at what's excluded. No cardfolder underscore id — you can't move a set to a different folder through update. No user underscore id — you can't transfer ownership. Those are architectural constraints enforced by the model itself.

CardsetResponse shows everything: id, cardfolder underscore id, user underscore id, name, description, is underscore public, created underscore at.

❑ And card models — the most complex?

CardCreate has cardset underscore id — required, because a card must belong to a set — plus eight content fields, all optional. Front text, front audio text, front audio, front image, and the same four for back.

CardUpdate has just the eight content fields, all optional. No cardset underscore id — you can't move a card to a different set.

CardResponse has everything: id, cardset underscore id, all eight content fields, created underscore at, and the audit fields — updated underscore at and updated underscore by, both optional. They're optional because a brand new card that's never been edited has null audit fields.

❑ The outline highlights a critical exclusion at point five. What is it?

Updated underscore at and updated underscore by appear only in CardResponse. Never in CardCreate. Never in CardUpdate. The API sets them automatically during updates. No client can provide them. No one can fake their audit trail. If they appeared in the Update model, a malicious user could stamp someone else's ID or backdate a timestamp. The model prevents it.

❑ Now, subsection D covers how the API identifies the requesting user. The outline describes a simplified authentication using the x-user-id header. How does that work?

The client sends their user ID as an HTTP header called x-user-id with every request. FastAPI reads it using the Header dependency. Two syntax forms. Header with an ellipsis — the header is required, missing it gives a 422 error. Header with default equals None — optional, value is None if missing. And there's a nice automatic conversion: Python uses underscores in variable names, but HTTP headers use hyphens. You write x underscore user underscore id in Python, the client sends x-user-id as the header. FastAPI converts between them automatically.

❑ But the outline warns this isn't secure for production.

Correct. Anyone can fake the header. There's no verification that the user ID actually belongs to the person sending the request. In production, you'd use JWT tokens, OAuth, or Supabase Auth. But the ownership and visibility logic — checking who you are, comparing you to the resource owner — is identical regardless of how you establish identity. We're learning the logic, not the authentication infrastructure.

❑ Section summary in the outline: Pydantic models in three categories, user underscore id never in Create models, is underscore public defaults to false, audit fields only in Response models, simplified auth via the header. Let's build the actual endpoints.

SECTION V — COMPLETE CODE

File: `models/user_models.py`

```
from pydantic import BaseModel
from datetime import datetime

class UserCreate(BaseModel):
    username: str
    email: str

class UserResponse(BaseModel):
    id: str
    username: str
    email: str
    created_at: datetime
```

File: `models/cardfolder_models.py`

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CardfolderCreate(BaseModel):
    name: str
    description: Optional[str] = None

class CardfolderResponse(BaseModel):
    id: str
    user_id: str
    name: str
    description: Optional[str] = None
    created_at: datetime
```

File: models/cardset_models.py

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CardsetCreate(BaseModel):
    cardfolder_id: str
    name: str
    description: Optional[str] = None
    is_public: bool = False

class CardsetUpdate(BaseModel):
    name: Optional[str] = None
    description: Optional[str] = None
    is_public: Optional[bool] = None

class CardsetResponse(BaseModel):
    id: str
    cardfolder_id: str
    user_id: str
    name: str
    description: Optional[str] = None
    is_public: bool
    created_at: datetime
```

File: models/card_models.py

```

from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CardCreate(BaseModel):
    cardset_id: str
    front_text_1: Optional[str] = None
    front_audio_text_1: Optional[str] = None
    front_audio_1: Optional[str] = None
    front_image: Optional[str] = None
    back_text_1: Optional[str] = None
    back_audio_text_1: Optional[str] = None
    back_audio_1: Optional[str] = None
    back_image: Optional[str] = None

class CardUpdate(BaseModel):
    front_text_1: Optional[str] = None
    front_audio_text_1: Optional[str] = None
    front_audio_1: Optional[str] = None
    front_image: Optional[str] = None
    back_text_1: Optional[str] = None
    back_audio_text_1: Optional[str] = None
    back_audio_1: Optional[str] = None
    back_image: Optional[str] = None

class CardResponse(BaseModel):
    id: str
    cardset_id: str
    front_text_1: Optional[str] = None
    front_audio_text_1: Optional[str] = None
    front_audio_1: Optional[str] = None
    front_image: Optional[str] = None
    back_text_1: Optional[str] = None
    back_audio_text_1: Optional[str] = None
    back_audio_1: Optional[str] = None
    back_image: Optional[str] = None
    created_at: datetime
    updated_at: Optional[datetime] = None
    updated_by: Optional[str] = None

```

Header dependency syntax (used across all routers):

```
from fastapi import Header

# Required header – request fails with 422 if missing:
def my_endpoint(x_user_id: str = Header(...)):

# Optional header – defaults to None if missing:
def my_endpoint(x_user_id: str = Header(default=None)):
```

SECTION VI — BUILDING EVERY ENDPOINT

□ Section VI — this is the main event. We're building every endpoint. How many are we talking about?

Twenty-one across four routers, plus one more copy endpoint in Section VIII for a total of twenty-two. If you look at the table in subsection A: Users — three endpoints. Folders — five. Card sets — seven. Cards — six. Let's take them one router at a time.

□ But first — the outline covers HTTP status codes in subsection B. Why does that matter?

Because the status code is how the API communicates what happened. It's the language of API responses. Every response carries a numeric code. 200 range means success. 400 range means the client made an error. 500 range means the server broke.

We use seven specific codes, and I want to highlight a distinction that most beginners get wrong. Look at the table in your outline. 401 Unauthorized versus 403 Forbidden.

□ They sound similar. What's the difference?

401 means "we don't know who you are." The user ID doesn't correspond to any registered user. Authentication failure — your identity is unknown.

403 means "we know who you are, but you're not allowed." You're a real, verified user, but you don't own this resource. Authorisation failure — identity known, permissions insufficient.

The other codes: 200 OK for successful reads, updates, deletes. 201 Created for successful creation. 404 Not Found when the resource doesn't exist. 409 Conflict when the data is valid but conflicts with current state — like a duplicate username. And 422 when Pydantic validation fails.

□ The outline also describes helper functions before the routers — four reusable verification functions in subsection C. Why?

Because ownership verification happens on almost every endpoint. Writing the same check inside twenty different functions is repetitive and error-prone. Instead, we extract it into helpers that every endpoint calls.

Verify underscore user takes a user ID, queries the users table. If the user doesn't exist — 401 Unauthorized.

Verify underscore folder underscore ownership takes a folder ID and user ID. Fetches the folder. Doesn't exist? 404. Exists but the user doesn't match? 403.

Verify underscore cardset underscore ownership — same pattern for card sets. 404 or 403.

❑ But cards don't have their own user underscore id. So how does card ownership verification work?

That's what the fourth helper solves — get underscore card underscore and underscore verify underscore ownership. It performs a two-step chain. First, fetch the card. If it doesn't exist — 404. Then look at which card set this card belongs to, and verify that the requesting user owns that card set. If not — 403. Two database lookups chained together, because card ownership is inherited, not direct.

❑ Alright. Let's go through the routers. User router first — three endpoints.

The simplest router. POST slash users — register a new user. Before inserting, the endpoint checks whether the username or email already exists. If either does — 409 Conflict with a clear message like "Username already taken."

❑ Why check explicitly? The database has UNIQUE constraints — wouldn't it catch duplicates?

It would, but the error message would be a raw database error — cryptic, unhelpful to the client. By checking explicitly first, we return a human-readable message. Better user experience, same protection.

GET slash users — list all registered users. No authentication required for this tutorial.

GET slash users slash user underscore id — get one specific user. Returns them or 404.

❑ Folder router — five endpoints. I'm looking at subsection E in the outline.

POST slash cardfolders — create a folder. Here's the critical line: data bracket user underscore id equals x underscore user underscore id. The ownership is set server-side, from the header. The client's request body doesn't contain user underscore id at all — remember, it's excluded from the Create model. The client literally cannot claim to create a folder on behalf of another user. The system controls who owns what.

□ That's the ownership enforcement pattern.

And it repeats everywhere. GET slash cardfolders slash my — returns only folders where user underscore id matches the requester. Your folders, nobody else's.

GET slash cardfolders slash folder underscore id — get one folder by ID. 404 if not found.

PUT slash cardfolders slash folder underscore id — update. Calls verify underscore user first, then verify underscore folder underscore ownership. Not the owner? 403 Forbidden. You cannot edit someone else's folder.

DELETE slash cardfolders slash folder underscore id — delete with cascades. Same ownership check. If you delete a folder, all its card sets are destroyed, and all the cards inside those sets are destroyed. The cascade chain we designed in Section III.

□ Card set router — seven endpoints. This is the biggest router. Let's take them in logical groups.

Let's start with creation and retrieval of your own stuff. POST slash cardsets — create a card set. Two verifications before anything happens: the requesting user must exist, AND the parent folder must exist and belong to that user. You can't create a card set inside someone else's folder.

GET slash cardsets slash my — all your card sets. Simple filter on user underscore id.

□ Now the public and visibility endpoints — this is where it gets interesting.

GET slash cardsets slash public — returns everything where is underscore public is true. No authentication required. Anyone — even an anonymous client with no header — can browse public sets.

GET slash cardsets slash cardset underscore id — get one specific card set. And this is where the visibility logic kicks in. Notice something about the header: it's optional here. Header with default equals None. Why?

❑ Because a public set should be viewable by anyone?

Exactly. If the set is public — return it, no matter who's asking, even if there's no header at all. If the set is private — now we check. Is the requester the owner? If yes, return it. If no, or if there's no header — 403 Forbidden. The same endpoint handles both public and private access with a single piece of branching logic.

❑ What about viewing card sets within a specific folder?

GET slash cardsets slash folder slash folder underscore id — gets all card sets in a folder, but filtered. It fetches everything from that folder, then filters in Python: keep sets that are either public OR owned by the requesting user. You see your own stuff plus anything public. Other people's private sets are invisible.

❑ Why filter in Python instead of in the database query?

Pragmatic reason. Supabase's chained query API doesn't easily support OR conditions like "where is underscore public equals true OR user underscore id equals X." For small to medium datasets, fetching all and filtering in Python is perfectly fine. For massive datasets, you'd use a stored procedure or Supabase's or underscore filter method. But for this tutorial, in-memory filtering works.

❑ Now the write operations — update and delete.

PUT slash cardsets slash cardset underscore id — update, owner only. The usual ownership verification. And here's a crucial detail: the code uses model underscore dump with exclude underscore unset equals True.

❑ What does that do?

It solves a subtle but devastating problem. When the client sends an update, they might only want to change the name — nothing else. Without exclude underscore unset, every field the client didn't mention gets sent as None to the database, overwriting existing data. Your carefully written description? Gone. Your is underscore public flag? Reset to None.

With `exclude underscore unset equals True`, only fields the client actually included in their request body get sent. "I didn't mention the description" means leave it unchanged. "I explicitly set the description to null" means clear it. The distinction matters, and `model underscore dump` handles it correctly.

`DELETE slash cardsets slash cardset underscore id` — owner only, cascades to all cards inside.

❑ Card router — six endpoints. Subsection G.

`POST slash cards` — create a card. Verifies the user exists and owns the parent card set. Can't sneak cards into someone else's set. And here's a nice touch: if the client provides `front underscore text underscore 1` but not `front underscore audio underscore text underscore 1`, the API automatically generates the audio text by calling a function we'll build in Section VII. Same for back text. The client gets TTS-ready text for free.

❑ What about reading cards? How does visibility work at the card level?

`GET slash cards slash card underscore id` — get one card. Card visibility is inherited from the parent card set. The endpoint looks up which set the card belongs to, checks whether that set is public. If it's public — anyone can see the card. If it's private — only the owner. Making a card set private hides all its cards automatically. No per-card flags needed.

`GET slash cards slash cardset slash cardset underscore id` — get all cards in a set. Same visibility check, applied once to the parent set.

❑ Now the update endpoint — this is where the audit trail happens, right?

`PUT slash cards slash card underscore id` — update, owner only, plus audit trail. First, ownership verification through the two-step chain — fetch the card, verify the parent card set's owner. If the client changes the text but doesn't provide new audio text, it auto-regenerates. And then the two critical lines.

`Data bracket updated underscore at equals datetime dot now with timezone dot utc, then dot isoformat`. That stamps the current UTC time as an ISO 8601 string — a format like `"2025-03-15T14:30:00+00:00"` that's unambiguous and machine-parseable across every language and system. UTC ensures every timestamp is in the same frame regardless of where the server runs.

Data bracket updated underscore by equals x underscore user underscore id.

Records who made the edit. Both fields from Python's built-in datetime module — no extra packages.

❑ And the remaining two endpoints?

DELETE slash cards slash card underscore id — owner only, two-step ownership chain.

GET slash cards slash cardset slash cardset underscore id slash updates — this is the audit history view. Instead of returning full cards with all content fields, it returns a lightweight view — just id, cardset underscore id, updated underscore at, and updated underscore by. Think of it as a dashboard: which cards were recently modified, and by whom. The SELECT call requests only those specific columns — more efficient for this overview purpose.

❑ Subsection H summarises key patterns across all routers. Let me pull them from the outline. Two authentication patterns — mandatory Header with ellipsis for write operations, optional Header with default for reads that allow public access. Consistent error codes — 401, 403, 404, 409, 422. Ownership enforcement on every single write operation: verify user, verify ownership, perform action. And visibility cascading from card set to cards. That's the entire endpoint layer.

SECTION VI — COMPLETE CODE

File: routers/user_router.py

```

from fastapi import APIRouter, HTTPException
from models.user_models import UserCreate, UserResponse
from database import supabase
from typing import List

router = APIRouter(
    prefix="/users",
    tags=["Users"]
)

# -----
# ENDPOINT 1: Register a new user
# -----
@router.post("/", response_model=UserResponse)
def create_user(user: UserCreate):
    # Check if username already exists
    existing = supabase.table("users").select("id").eq("username",
user.username).execute()
    if existing.data:
        raise HTTPException(status_code=409, detail="Username already
taken")

    # Check if email already exists
    existing_email = supabase.table("users").select("id").eq("email",
user.email).execute()
    if existing_email.data:
        raise HTTPException(status_code=409, detail="Email already
registered")

    data = user.model_dump()
    result = supabase.table("users").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create
user")

    return result.data[0]

```

```

# -----
# ENDPOINT 2: Get all users
# -----
@router.get("/", response_model=List[UserResponse])
def get_all_users():
    result = supabase.table("users").select("*").execute()
    return result.data

# -----
# ENDPOINT 3: Get one user by ID
# -----
@router.get("/{user_id}", response_model=UserResponse)
def get_user(user_id: str):
    result = supabase.table("users").select("*").eq("id",
user_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="User not found")

    return result.data[0]

```

File: routers/cardfolder_router.py

```

from fastapi import APIRouter, HTTPException, Header
from models.cardfolder_models import CardfolderCreate, CardfolderResponse
from database import supabase
from typing import List

router = APIRouter(
    prefix="/cardfolders",
    tags=["Card Folders"]
)

# -----
# HELPER: Verify the requesting user exists in the database
# -----
def verify_user(user_id: str):
    result = supabase.table("users").select("id").eq("id",
user_id).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="User not found -
invalid x-user-id header")

# -----
# HELPER: Verify the requesting user owns the given folder
# -----
def verify_folder_ownership(folder_id: str, user_id: str):
    result = supabase.table("cardfolders").select("*").eq("id",
folder_id).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Folder not found")
    folder = result.data[0]
    if folder["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="You do not own this
folder")
    return folder

# -----
# ENDPOINT 1: Create a new folder

```

```

# -----
@router.post("/", response_model=CardfolderResponse)
def create_cardfolder(folder: CardfolderCreate, x_user_id: str =
Header(...)):
    verify_user(x_user_id)

    data = folder.model_dump()
    data["user_id"] = x_user_id # Set the owner to the requesting user

    result = supabase.table("cardfolders").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create
folder")

    return result.data[0]

# -----
# ENDPOINT 2: Get all folders owned by me
# -----
@router.get("/my", response_model=List[CardfolderResponse])
def get_my_cardfolders(x_user_id: str = Header(...)):
    verify_user(x_user_id)
    result = supabase.table("cardfolders").select("*").eq("user_id",
x_user_id).execute()
    return result.data

# -----
# ENDPOINT 3: Get one folder by ID
# -----
@router.get("/{folder_id}", response_model=CardfolderResponse)
def get_cardfolder(folder_id: str):
    result = supabase.table("cardfolders").select("*").eq("id",
folder_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Folder not found")

```

```

        return result.data[0]

# -----
# ENDPOINT 4: Update a folder (owner only)
# -----
@router.put("/{folder_id}", response_model=CardfolderResponse)
def update_cardfolder(folder_id: str, folder: CardfolderCreate, x_user_id:
str = Header(...)):
    verify_user(x_user_id)
    verify_folder_ownership(folder_id, x_user_id)

    data = folder.model_dump()
    result = supabase.table("cardfolders").update(data).eq("id",
folder_id).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to update
folder")

    return result.data[0]

# -----
# ENDPOINT 5: Delete a folder (owner only)
# -----
@router.delete("/{folder_id}")
def delete_cardfolder(folder_id: str, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    verify_folder_ownership(folder_id, x_user_id)

    result = supabase.table("cardfolders").delete().eq("id",
folder_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Folder not found")

    return {"message": "Folder deleted successfully", "deleted":

```

```
result.data[0]}
```

File: routers/cardset_router.py

```

from fastapi import APIRouter, HTTPException, Header
from models.cardset_models import CardsetCreate, CardsetUpdate,
CardsetResponse
from database import supabase
from typing import List

router = APIRouter(
    prefix="/cardsets",
    tags=["Card Sets"]
)

# -----
# HELPER: Verify the requesting user exists in the database
# -----
def verify_user(user_id: str):
    result = supabase.table("users").select("id").eq("id",
user_id).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="User not found –
invalid x-user-id header")

# -----
# HELPER: Verify the requesting user owns the given card set
# Returns the card set data if ownership is confirmed
# -----
def verify_ownership(cardset_id: str, user_id: str):
    result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Card set not found")
    cardset = result.data[0]
    if cardset["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="You do not own this
card set")
    return cardset

```

```

# -----
# ENDPOINT 1: Create a new card set
# -----
@router.post("/", response_model=CardsetResponse)
def create_cardset(cardset: CardsetCreate, x_user_id: str = Header(...)):
    verify_user(x_user_id)

    # Verify the parent folder exists and belongs to this user
    folder_check = supabase.table("cardfolders").select("*").eq("id",
cardset.cardfolder_id).execute()
    if not folder_check.data:
        raise HTTPException(status_code=404, detail="Parent folder not
found")
    if folder_check.data[0]["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="You do not own the
parent folder")

    data = cardset.model_dump()
    data["user_id"] = x_user_id # Set the owner to the requesting user

    result = supabase.table("cardsets").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create card
set")

    return result.data[0]

# -----
# ENDPOINT 2: Get all card sets owned by me
# -----
@router.get("/my", response_model=List[CardsetResponse])
def get_my_cardsets(x_user_id: str = Header(...)):
    verify_user(x_user_id)
    result = supabase.table("cardsets").select("*").eq("user_id",
x_user_id).execute()
    return result.data

```

```

# -----
# ENDPOINT 3: Get all public card sets
# -----
@router.get("/public", response_model=List[CardsetResponse])
def get_public_cardsets():
    result = supabase.table("cardsets").select("*").eq("is_public",
True).execute()
    return result.data

# -----
# ENDPOINT 4: Get one card set by ID (visibility enforced)
# -----
@router.get("/{cardset_id}", response_model=CardsetResponse)
def get_cardset(cardset_id: str, x_user_id: str = Header(default=None)):
    result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    cardset = result.data[0]

    # If the set is private, only the owner can see it
    if not cardset["is_public"] and cardset["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="This card set is
private")

    return cardset

# -----
# ENDPOINT 5: Get card sets in a folder (visibility enforced)
# -----
@router.get("/folder/{folder_id}", response_model=List[CardsetResponse])
def get_cardsets_by_folder(folder_id: str, x_user_id: str =
Header(default=None)):
    result = supabase.table("cardsets").select("*").eq("cardfolder_id",

```

```

folder_id).execute()

    # Filter: show public sets + sets owned by the requesting user
    visible = []
    for cs in result.data:
        if cs["is_public"] or cs["user_id"] == x_user_id:
            visible.append(cs)

    return visible

# -----
# ENDPOINT 6: Update a card set (owner only)
# -----
@router.put("/{cardset_id}", response_model=CardsetResponse)
def update_cardset(cardset_id: str, cardset: CardsetUpdate, x_user_id: str
= Header(...)):
    verify_user(x_user_id)
    verify_ownership(cardset_id, x_user_id)

    data = cardset.model_dump(exclude_unset=True)

    if not data:
        raise HTTPException(status_code=400, detail="No fields to update")

    result = supabase.table("cardsets").update(data).eq("id",
cardset_id).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to update card
set")

    return result.data[0]

# -----
# ENDPOINT 7: Delete a card set (owner only)
# -----
@router.delete("/{cardset_id}")

```

```
def delete_cardset(cardset_id: str, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    verify_ownership(cardset_id, x_user_id)

    result = supabase.table("cardsets").delete().eq("id",
cardset_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    return {"message": "Card set deleted successfully", "deleted":
result.data[0]}
```

File: routers/card_router.py

```

from fastapi import APIRouter, HTTPException, Header
from models.card_models import CardCreate, CardUpdate, CardResponse
from database import supabase
from utils.audio_text import generate_audio_text
from typing import List
from datetime import datetime, timezone

router = APIRouter(
    prefix="/cards",
    tags=["Cards"]
)

# -----
# HELPER: Verify user exists
# -----
def verify_user(user_id: str):
    result = supabase.table("users").select("id").eq("id",
user_id).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="User not found –
invalid x-user-id header")

# -----
# HELPER: Verify the requesting user owns the card set
# that a card belongs to (or will belong to)
# -----
def verify_cardset_ownership(cardset_id: str, user_id: str):
    result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Parent card set not
found")
    if result.data[0]["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="You do not own the
parent card set")
    return result.data[0]

```

```

# -----
# HELPER: Given a card ID, look up which cardset it belongs to
# and verify ownership. Returns the card data.
# -----
def get_card_and_verify_ownership(card_id: str, user_id: str):
    card_result = supabase.table("cards").select("*").eq("id",
card_id).execute()
    if not card_result.data:
        raise HTTPException(status_code=404, detail="Card not found")
    card = card_result.data[0]
    verify_cardset_ownership(card["cardset_id"], user_id)
    return card

# -----
# ENDPOINT 1: Create a new card
# -----
@router.post("/", response_model=CardResponse)
def create_card(card: CardCreate, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    verify_cardset_ownership(card.cardset_id, x_user_id)

    data = card.model_dump()

    # Auto-generate audio text if not provided
    if data.get("front_text_1") and not data.get("front_audio_text_1"):
        data["front_audio_text_1"] =
generate_audio_text(data["front_text_1"])

    if data.get("back_text_1") and not data.get("back_audio_text_1"):
        data["back_audio_text_1"] =
generate_audio_text(data["back_text_1"])

    result = supabase.table("cards").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create
card")

```

```

        return result.data[0]

# -----
# ENDPOINT 2: Get one card by ID
# -----
@router.get("/{card_id}", response_model=CardResponse)
def get_card(card_id: str, x_user_id: str = Header(default=None)):
    card_result = supabase.table("cards").select("*").eq("id",
card_id).execute()
    if not card_result.data:
        raise HTTPException(status_code=404, detail="Card not found")

    card = card_result.data[0]

    # Check visibility of the parent card set
    cardset = supabase.table("cardsets").select("*").eq("id",
card["cardset_id"]).execute()
    if cardset.data:
        cs = cardset.data[0]
        if not cs["is_public"] and cs["user_id"] != x_user_id:
            raise HTTPException(status_code=403, detail="This card belongs
to a private card set")

    return card

# -----
# ENDPOINT 3: Get all cards in a card set (visibility enforced)
# -----
@router.get("/cardset/{cardset_id}", response_model=List[CardResponse])
def get_cards_by_cardset(cardset_id: str, x_user_id: str =
Header(default=None)):
    # Check that the card set exists and is visible
    cs_result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not cs_result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

```

```

    cs = cs_result.data[0]
    if not cs["is_public"] and cs["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="This card set is
private")

    result = supabase.table("cards").select("*").eq("cardset_id",
cardset_id).execute()
    return result.data

# -----
# ENDPOINT 4: Update a card (owner only, writes audit fields)
# -----
@router.put("/{card_id}", response_model=CardResponse)
def update_card(card_id: str, card: CardUpdate, x_user_id: str =
Header(...)):
    verify_user(x_user_id)
    get_card_and_verify_ownership(card_id, x_user_id)

    data = card.model_dump(exclude_unset=True)

    # Regenerate audio text if display text changed but audio text not
explicitly set
    if "front_text_1" in data and "front_audio_text_1" not in data:
        if data["front_text_1"] is not None:
            data["front_audio_text_1"] =
generate_audio_text(data["front_text_1"])
        else:
            data["front_audio_text_1"] = None

    if "back_text_1" in data and "back_audio_text_1" not in data:
        if data["back_text_1"] is not None:
            data["back_audio_text_1"] =
generate_audio_text(data["back_text_1"])
        else:
            data["back_audio_text_1"] = None

    if not data:

```

```

        raise HTTPException(status_code=400, detail="No fields to update")

# — AUDIT TRAIL: Write update timestamp and user —
data["updated_at"] = datetime.now(timezone.utc).isoformat()
data["updated_by"] = x_user_id

result = supabase.table("cards").update(data).eq("id",
card_id).execute()

if not result.data:
    raise HTTPException(status_code=400, detail="Failed to update
card")

return result.data[0]

# —————
# ENDPOINT 5: Delete a card (owner only)
# —————
@router.delete("/{card_id}")
def delete_card(card_id: str, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    get_card_and_verify_ownership(card_id, x_user_id)

    result = supabase.table("cards").delete().eq("id", card_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Card not found")

    return {"message": "Card deleted successfully", "deleted":
result.data[0]}

# —————
# ENDPOINT 6: Get update history for all cards in a card set
# Shows each card's last update timestamp and who updated it
# —————
@router.get("/cardset/{cardset_id}/updates")
def get_card_updates(cardset_id: str, x_user_id: str =

```

```
Header(default=None)):
    # Visibility check
    cs_result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not cs_result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    cs = cs_result.data[0]
    if not cs["is_public"] and cs["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="This card set is
private")

    result = supabase.table("cards").select(
        "id, front_text_1, back_text_1, created_at, updated_at, updated_by"
    ).eq("cardset_id", cardset_id).execute()

    return result.data
```

SECTION VII — AUDIO TEXT GENERATION WITH REGULAR EXPRESSIONS

❑ Section VII — audio text generation. We mentioned earlier that display text and speech text are different. Let me understand the problem concretely.

Take a flashcard for learning Spanish. On screen, it shows: "El gato (the cat) [m.]" That's perfectly clear visually. "El gato" is the word. "(the cat)" is the English translation hint. "[m.]" tells you it's masculine. A human reader processes these three pieces instantly.

Now send that text to a text-to-speech engine. TTS engines read literally — every character, every punctuation mark. What you hear is: "El gato open parenthesis the cat close parenthesis open bracket m period close bracket." Unintelligible. Useless for language learning.

❑ So we need to strip out the visual formatting and keep just the speakable part.

Exactly. "El gato (the cat) [m.]" needs to become "El gato." Clean, natural, ready for speech. And we need to do this automatically, because asking users to manually create audio-friendly text for every card defeats the purpose.

❑ What tool do we use?

Regular expressions — regex. Specifically, Python's `re dot sub` function. If you haven't used regex before, here's the idea: instead of searching for a specific string like "find the word cat," a regex describes a pattern — like "find anything between square brackets." Python's `re` module provides the functions, and `re dot sub` is for substitution. Three arguments: the pattern to find, the replacement — often an empty string, which effectively deletes the match — and the text to search within. Returns a new string.

❑ The outline lists five transformation rules in subsection C, applied in a specific order. Why does order matter?

Because each rule leaves behind artifacts that later rules clean up. If you run the cleanup step first, it has nothing to clean — and then the other rules create new messes. The sequence is deliberate. Let me walk through each rule, and then we'll trace a complete example.

❑ Rule one.

Remove square bracket content. The pattern `backslash bracket dot star question mark backslash bracket` matches an opening bracket, then any characters — as few as possible — then a closing bracket. Replace with empty string. "El gato [m.]" becomes "El gato" with a trailing space where the brackets were.

❑ Hold on — "as few as possible." What's the question mark doing?

This is the greedy versus non-greedy distinction, and it's critical. By default, regex is greedy — dot star matches as many characters as possible. If your text is "[one] word [two]" and you use a greedy pattern, it matches from the first opening bracket all the way to the last closing bracket — "[one] word [two]" as a single match. The word between them gets swallowed.

The question mark makes it non-greedy — match as few characters as possible. Now it matches "[one]" first, then "[two]" separately. The word between is preserved. For flashcards that might have multiple bracketed annotations, non-greedy is essential.

❑ Rule two.

Remove parenthetical content. Same approach, same non-greedy matching. "El gato (the cat)" becomes "El gato" with a trailing space.

❑ Rule three — this one's different.

Right. Language flashcards sometimes show alternatives with a slash — "rojo/a" meaning the masculine form "rojo" or the feminine form "roja." For audio, we want only the first form.

This rule uses a capture group. Parentheses in the regex capture a portion of the match. The pattern is `backslash w plus` — that's one or more word characters — inside parentheses to capture it, then a slash, then another `backslash w plus`. The replacement is `backslash 1` — a backreference meaning "put back whatever was in the first capture group."

❑ Wait — how does Python handle that backslash 1?

In Python, you write it as `r backslash 1` — with an `r` prefix making it a raw string. And this matters. Without the `r` prefix, Python's string parser would interpret backslash 1 as a special character — specifically, the character with ASCII value 1 — before the regex engine ever sees it. The raw string prefix tells Python to pass the backslash through untouched. The regex engine receives the literal backslash 1 and interprets it as a backreference. Two different layers of interpretation — you need the raw string to bypass the first one.

So `"rojo/a"` becomes `"rojo."` The first form is kept, the slash and everything after it is discarded.

❑ Rule four.

Remove leading numbers. Matches digits followed by a dot at the start of the string. `"3. El gato"` becomes `"El gato"` with a leading space.

❑ And rule five — the cleanup.

This must be last. Every previous rule removed content but left gaps — extra spaces where brackets, parentheses, slashes, and numbers used to be. Rule five collapses multiple consecutive spaces into a single space and strips leading and trailing whitespace. If you ran this earlier, the gaps from subsequent rules would remain uncleaned.

❑ Let's trace the complete example from the outline. Subsection D has a table — I'm looking at it now.

Input: `"3. El gato rojo/a (the red cat) [m./f.]"`

After rule one — brackets gone: `"3. El gato rojo/a (the red cat)"` with trailing space.

After rule two — parentheses gone: `"3. El gato rojo/a"` with extra spaces.

After rule three — slash simplified: `"3. El gato rojo"` with extra spaces.

After rule four — leading number gone: `"El gato rojo"` with leading and extra spaces.

After rule five — whitespace cleaned: `"El gato rojo."`

Clean, natural, ready for TTS. From a visually rich flashcard string to a speakable sentence, automatically.

❑ How does this integrate with the card router?

The function is called automatically when a client provides display text — front underscore text underscore 1 or back underscore text underscore 1 — but does not provide the corresponding audio text field. If they don't supply front underscore audio underscore text underscore 1, the API generates it. If they do supply it — their explicit value takes priority. Auto-generation is a default, not a mandate.

And it's extensible. Adding a new rule is just another re dot sub line at the appropriate position — keeping the whitespace cleanup at the end.

❑ Section summary from the outline: five rules in strict order, non-greedy matching prevents overmatching, capture groups and backreferences for partial replacement, whitespace cleanup last, auto-generation is automatic but overridable. Onto the final major feature.

SECTION VII — COMPLETE CODE

File: `utils/audio_text.py`

```

import re

def generate_audio_text(display_text: str) → str:
    """
    Transform display text into audio-friendly text by removing
    annotations, hints, and formatting that would sound unnatural
    when read aloud by a text-to-speech engine.

    Rules applied in order:
    1. Remove content in square brackets: [m.] [noun] [informal]
    2. Remove content in parentheses: (the cat) (lit. house)
    3. Simplify slash alternatives to first word: rojo/a → rojo
    4. Remove leading numbering: 1. 2. 3.
    5. Clean up whitespace
    """
    text = display_text

    # Rule 1: Remove square bracket content
    # \[ matches a literal opening bracket
    # .*? matches any characters, non-greedy (as few as possible)
    # \] matches a literal closing bracket
    text = re.sub(r'\[.*?\]', '', text)

    # Rule 2: Remove parenthetical content
    # \( matches a literal opening parenthesis
    # .*? matches any characters, non-greedy
    # \) matches a literal closing parenthesis
    text = re.sub(r'\(.*?\)', '', text)

    # Rule 3: Simplify slash alternatives – keep only first word
    # (\w+) captures one or more word characters (the part we keep)
    # / matches a literal slash
    # \w+ matches one or more word characters (the part we discard)
    # \1 in the replacement refers to the first captured group
    text = re.sub(r'(\w+)/\w+', r'\1', text)

    # Rule 4: Remove leading numbering like "1. " or "23. "
    # ^ anchors to the start of the string

```

```
# \d+    matches one or more digits
# \.     matches a literal dot
# \s*    matches zero or more whitespace characters
text = re.sub(r'^\d+\.\s*', '', text)

# Rule 5: Collapse multiple spaces into one
# \s+    matches one or more whitespace characters
text = re.sub(r'\s+', ' ', text)

# Final strip to remove leading/trailing whitespace
text = text.strip()

return text
```

SECTION VIII — PUBLIC SHARING AND THE COPY FEATURE

❑ Section VIII — the copy feature. We've built public visibility. We've built ownership. Now what happens when those two collide? A student finds a great public card set but can't do anything with it except look.

Exactly the problem. Picture this — and the outline has this scenario in subsection A. A Spanish teacher spends hours building "DELE B2 Vocabulary" — 200 carefully crafted cards with audio text, image URLs, everything polished. She marks it public. Students find it, love it. But a student wants to personalise — add notes, tweak translations, remove cards they already know, maybe rearrange.

The student can't edit the original. They're not the owner. But even if we gave them permission — think about what happens. Their changes affect every other student using the same set. One person's customisation destroys the experience for everyone else.

❑ So we need a way to create a personal copy.

A deep copy. Not just the card set metadata — every single card inside, duplicated into the student's own account. Their copy, their property, completely independent from the original. The teacher's set untouched.

❑ The outline describes a five-step process in subsection B. Walk me through it.

Step one — verify the source card set exists and is accessible. It must be either public or owned by the requester. Yes, you can copy your own sets — useful for creating variations.

Step two — the user specifies a target folder via a query parameter called `target_underscore_folder_underscore_id`. That folder must exist and must belong to the requesting user. You can't copy into someone else's folder.

Step three — create a new card set record. Name is "Copy of" followed by the original name. Owner is the requester. Folder is the target folder. `is_public` is false — your copy starts private.

Step four — fetch all cards from the source card set.

Step five — duplicate every card into the new set.

❑ What exactly gets preserved, and what starts fresh?

If you look at the table in your outline — subsection B point two — there's a clear split. Preserved exactly: all eight content fields. Front text, front audio text, front audio, front image, and the same four for back. The educational content crosses over perfectly.

Starting fresh: the id — each card gets a new UUID. The cardset underscore id — points to the new set, not the original. Created underscore at — set to now by the database. Updated underscore at — null, because this copied card has never been edited, only created. Updated underscore by — null, no edits have occurred.

❑ The endpoint itself — what does the URL look like?

POST slash cardsets slash cardset underscore id slash copy, with target underscore folder underscore id as a query parameter. POST because we're creating new data. The cardset underscore id in the URL is the source. The query parameter tells us where the copy goes.

❑ The outline mentions a defensive programming pattern — dot get versus bracket access. What's that about?

When reading fields from the source cards to build the copies, the code uses card dot get with the field name instead of bracket access. If a field doesn't exist or isn't populated — which is valid, not every card has all eight fields — dot get returns None gracefully. Bracket access would crash with a KeyError. It's defensive: handle partial data without special-casing each field.

❑ And there's a performance consideration?

A big one. Instead of inserting cards one by one — one database call per card — all the new cards are collected in a list and inserted with a single call. Supabase's insert method accepts a list of dictionaries. Copying 200 cards? One network call, not 200. Over a network with any latency at all, that's the difference between a fraction of a second and an unacceptable wait.

❑ The outline has a full end-to-end test scenario in subsection D. Can we walk through it? I want to see the whole system working together.

Let's trace it. Two users, two folders, and we'll watch ownership, visibility, copying, and audit trails all interact.

First, we register two users. Teacher underscore ana and student underscore bob. Each gets a user ID back. Then teacher creates a folder called "Languages," student creates "My Studies."

Now teacher creates a card set called "Spanish Basics" inside her Languages folder, with is underscore public set to true. She creates a card — front text "El gato (the cat) [m.]," back text "The cat." The API auto-generates the audio text: "El gato." All the regex rules we just discussed, working automatically.

□ Now the student enters the picture.

The student calls GET slash cardsets slash public — no header needed. There's "Spanish Basics" in the results. The student copies it: POST slash cardsets slash the teacher's set ID slash copy, with target folder set to the student's "My Studies" folder. Response: "Copy of Spanish Basics," owned by the student, private.

□ And now the student can personalise it.

The student calls GET slash cardsets slash my — "Copy of Spanish Basics" appears. Gets all the cards from the new set. Picks one and updates it — changes front text to "El gato gordo (the fat cat)." The API stamps updated underscore at with the current timestamp, updated underscore by with the student's ID. And it auto-regenerates the audio text as "El gato gordo."

□ Here's the critical test — is the teacher's original untouched?

GET the cards from the original set. Every single card is exactly as the teacher left it. No modifications, no timestamps, no sign that a copy ever happened. The two sets are completely independent.

And if the student tries to directly edit one of the teacher's cards? PUT on the teacher's card ID with the student's header in place. 403 Forbidden: "You do not own the parent card set." The ownership system works exactly as designed.

□ Let me read the section summary in the outline... deep copy, content preserved, metadata fresh, only public or owned sets copyable, target folder must be yours, batch insert for performance, complete independence. That's a proper multi-user platform. One more section to go.

SECTION VIII — COMPLETE CODE

Add to `routers/cardset_router.py` **— the copy endpoint (add this import at the top alongside existing imports):**

```
from utils.audio_text import generate_audio_text
```

Then add the endpoint at the bottom of the file:

```

# -----
# ENDPOINT 8: Copy a public card set into your own account
# -----
@router.post("/{cardset_id}/copy", response_model=CardsetResponse)
def copy_cardset(cardset_id: str, target_folder_id: str, x_user_id: str =
Header(...)):
    verify_user(x_user_id)

    # Step 1: Fetch the source card set and verify it is accessible
    source_result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not source_result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    source = source_result.data[0]

    # Allow copy if: the set is public, OR the requester is the owner
    if not source["is_public"] and source["user_id"] != x_user_id:
        raise HTTPException(
            status_code=403,
            detail="This card set is private and you are not the owner"
        )

    # Step 2: Verify the target folder exists and belongs to the
    requesting user
    folder_check = supabase.table("cardfolders").select("*").eq("id",
target_folder_id).execute()
    if not folder_check.data:
        raise HTTPException(status_code=404, detail="Target folder not
found")
    if folder_check.data[0]["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="You do not own the
target folder")

    # Step 3: Create the new card set (owned by requesting user, private
    by default)
    new_cardset_data = {
        "cardfolder_id": target_folder_id,
        "user_id": x_user_id,

```

```

        "name": f"Copy of {source['name']}",
        "description": source.get("description"),
        "is_public": False
    }

    new_cardset_result =
supabase.table("cardsets").insert(new_cardset_data).execute()
    if not new_cardset_result.data:
        raise HTTPException(status_code=400, detail="Failed to create card
set copy")

    new_cardset = new_cardset_result.data[0]

    # Step 4: Fetch all cards from the source card set
    source_cards = supabase.table("cards").select("*").eq("cardset_id",
cardset_id).execute()

    # Step 5: Duplicate each card into the new card set
    if source_cards.data:
        new_cards = []
        for card in source_cards.data:
            new_card = {
                "cardset_id": new_cardset["id"],
                "front_text_1": card.get("front_text_1"),
                "front_audio_text_1": card.get("front_audio_text_1"),
                "front_audio_1": card.get("front_audio_1"),
                "front_image": card.get("front_image"),
                "back_text_1": card.get("back_text_1"),
                "back_audio_text_1": card.get("back_audio_text_1"),
                "back_audio_1": card.get("back_audio_1"),
                "back_image": card.get("back_image"),
            }
            new_cards.append(new_card)

    # Batch insert all cards at once for efficiency
    supabase.table("cards").insert(new_cards).execute()

    return new_cardset

```

SECTION IX — FINAL SUMMARY

❑ Section IX — the final summary. Let's take stock. What have we built?

If you look at subsection A in your outline, there's a table listing every file. Thirteen source files plus three init files. Dot env and dot gitignore for configuration. Database dot py for the Supabase connection. Main dot py as the orchestrator. Four model files, four router files, one audio text utility. Twenty-two endpoints plus one root health check.

❑ And the multi-user features — subsection B has the complete list.

Nine items. User registration and lookup. Folder ownership with cascading deletes. Card set ownership — same pattern. Ownership enforcement on every write operation across every entity. Public/private visibility through the is underscore public flag. Visibility filtering on every read operation. Card-level audit trail. Deep copy with batch duplication. Simplified authentication via the x-user-id header.

❑ How do we actually run it?

Three commands, and they're in your outline at subsection C. cd flashcard-api. workon vintel. uvicorn main colon app dash dash reload. Open localhost colon 8000 slash docs in your browser and every endpoint is there, interactive, testable.

❑ The outline closes with four things to remember above all else. Let's end with those — they're in subsection D.

First — the data hierarchy. Users, cardfolders, cardsets, cards. Ownership flows down from users. Visibility is controlled at the card set level and cascades to all cards within. Not at the folder level. Not at the card level. The card set level.

Second — CASCADE versus SET NULL. CASCADE means the child can't exist without the parent — destroy them together. SET NULL means the reference is informational — clear it, but the child survives. Every foreign key in this project uses one or the other, and the choice is never arbitrary.

Third — the API controls the audit trail, not the client. Updated underscore at and updated underscore by are set automatically by the server during updates. They never appear in Create or Update request bodies. No one can fake when or by whom an edit was made.

Fourth — deep copy means complete independence. Content is preserved exactly. IDs, timestamps, audit fields all start fresh. Edits to the copy never affect the original. Edits to the original never affect the copy. Two independent objects that happen to have the same content at the moment of creation.

□ And that's the whole outline — nine sections, four database tables, twenty-two endpoints, a regex engine, and a multi-user platform. If you've been following along with the printed outline, every principle, every table, every bullet point has been discussed. Go back to any section that needs another pass. The outline and this podcast are built to work together.

Thanks for sticking with us. Go build something.